

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name : Ashwin S

Reg no : 24011101007

Class : AI DS - A (batch 1)

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. The perceptron learning algorithm is implemented without relying on high-level machine learning libraries, the role of the activation function is studied, the learning process is visualized across epochs, and the classifier is evaluated on a real-world dataset using standard performance metrics.

2. Tasks Performed

1. Downloaded and understood the dataset.
2. Performed Exploratory Data Analysis (EDA).
3. Preprocessed the dataset (standardization, train/test split).
4. Implemented a Single Layer Perceptron from scratch using NumPy.
5. Trained the model using the perceptron learning rule.
6. Evaluated the classifier using standard performance metrics.
7. Analyzed the effect of different learning rates.
8. Compared the implementation with Scikit-learn's Perceptron.

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

In this implementation, weights were initialized from a small Gaussian distribution ($\mathcal{N}(0, 0.01^2)$) rather than zeros, and the bias was initialized to 0. Updates were performed online – i.e. sample by sample within each epoch – following the classical Rosenblatt rule shown above.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output: Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Experimental Results

The single layer perceptron model was trained on the Banknote Authentication dataset for 50 epochs at a learning rate of $\eta = 0.01$. The first epoch misclassified 53 samples, which reduced to just 12 at the end of 50th epoch. The model does not converge within 50 epochs, indicating that the two classes are close to linearly separable but actually is non-linearly separable.

The final weights vector after learning was $[-0.1931, -0.2132, -0.2176, -0.0217]$ (corresponding to variance, skewness, curtosis, and entropy respectively), and the final bias was -0.1000 . The magnitudes of these weights indicate that variance, skewness, and curtosis contributed most strongly to the decision boundary, while entropy contributed the least.

In the test set of 275 samples, the trained model achieved an accuracy of 97.81%, a precision of 98.37%, a recall of 96.80% and an F1-score of 0.9758. The confusion matrix shows 147 true negatives, 121 true positives, 2 false positives, and 4 false negatives. All learning rates among $\eta \in \{0.001, 0.01, 0.1\}$ were tested, representing the trade-off between convergence speed and stability, and produced similar results to builtin Perceptron (accuracy of 0.9781). This indicates the correctness of manual implementation.

8. Mandatory Plots and Interpretations

All plots were generated in vector `.eps` format at 600 DPI quality.

8.1 Feature Histograms

Feature Histograms (split by class)

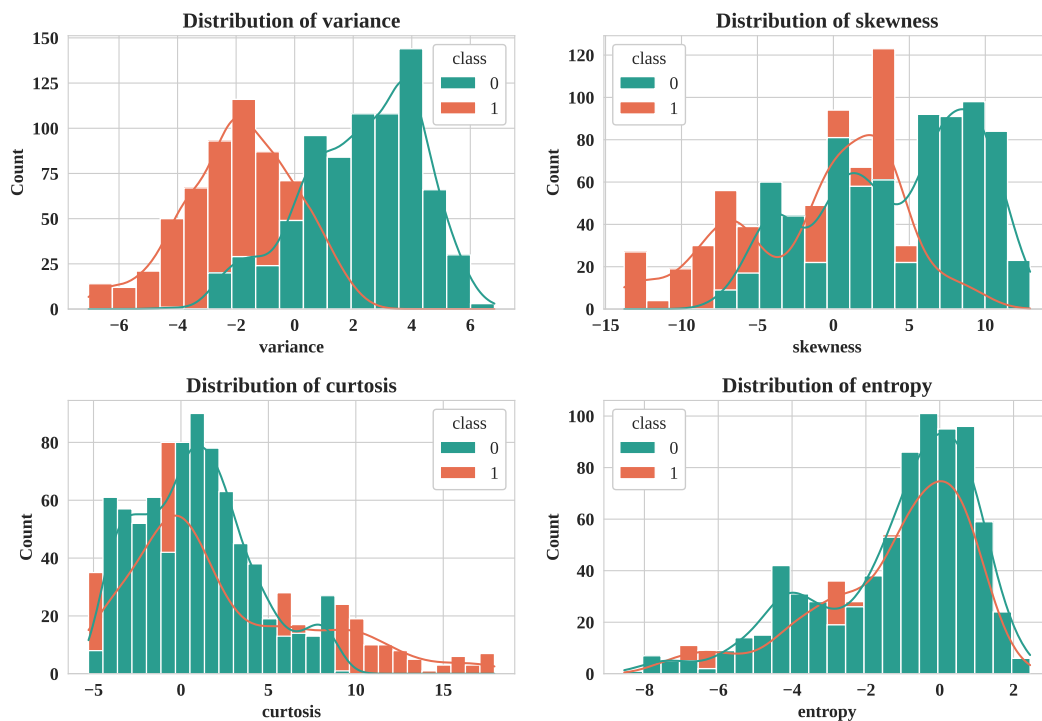


Figure 1: Distribution of each feature, split by class.

Variance shows clearly separated distribution between the two classes. Authentic notes tend to have higher variance, forged notes have lower variance. Entropy shows heavy overlap between classes, making it the weakest differentiating feature. Kurtosis shows a long right tail with partial overlap.

8.2 Correlation Heatmap

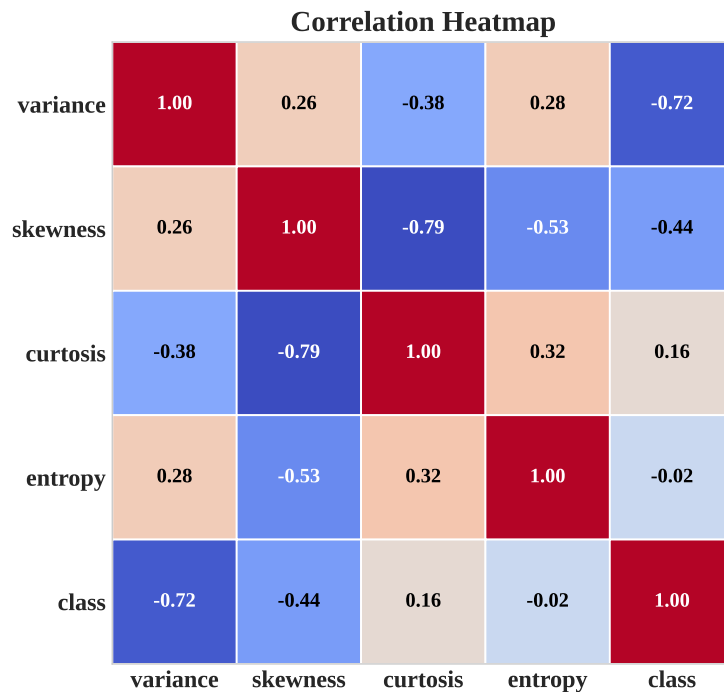


Figure 2: Correlation heatmap of the four features and the class label.

Variance has the strongest correlation with class. Skewness and kurtosis show moderate correlation with each other, which might be indicating some feature redundancy. Entropy correlates weakly with class, similar to its overlapping in the histogram plot.

8.3 Scatter Plot

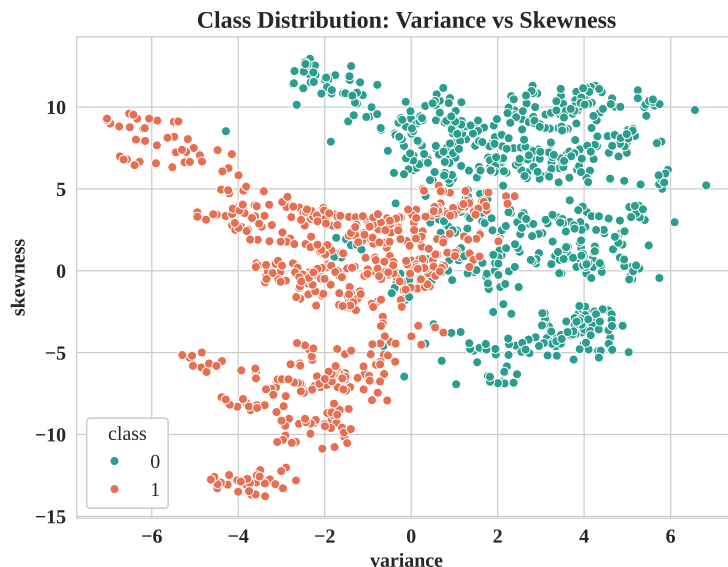


Figure 3: Class distribution: variance vs. skewness.

The two classes form moderately distinct clusters, separated by a rough diagonal boundary. Few points near the boundaries are inside the wrong clusters. However, since there is no clear separation, this data is not linearly separable.

8.4 Boxplots

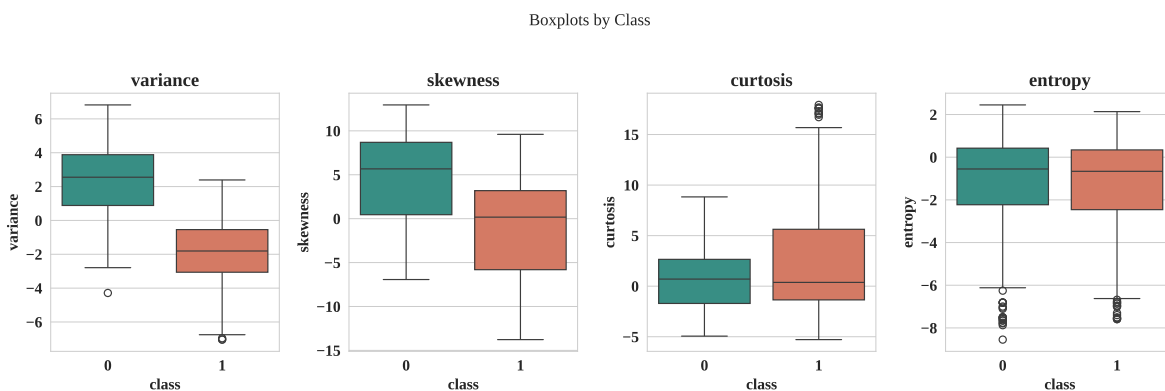


Figure 4: Boxplots of each feature by class.

Variance and skewness boxplots show minimal overlap in interquartile ranges between classes and the medians are clearly separated. Kurtosis shows moderate separation and multiple outliers on authentic notes. Entropy's boxes almost completely overlap, consistent with its poor correlation earlier.

8.5 Training Error vs Epoch



Figure 5: Number of misclassified training samples per epoch.

Misclassification count drops sharply within the first several epochs, then stabilizes and slightly declines. It started at 53 misclassified samples and ended at just 12. It could not reach further optimization as the data is non-linearly separable.

8.6 Weight Evolution

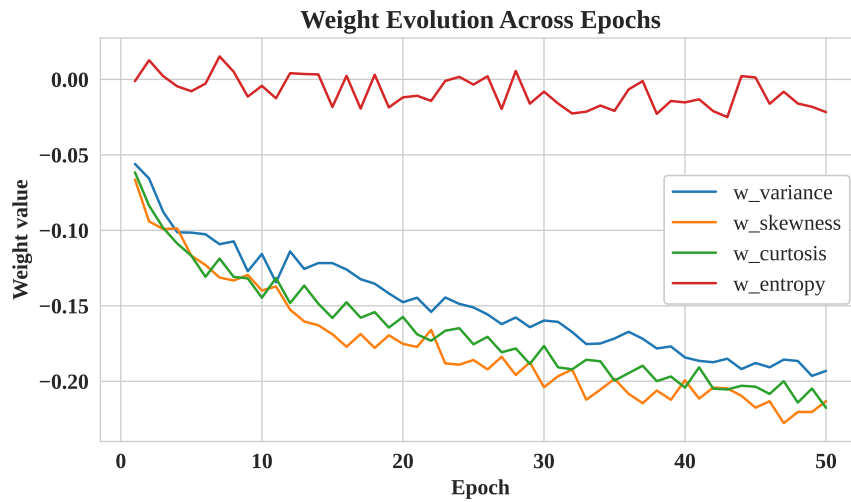


Figure 6: Evolution of the four feature weights across training epochs.

Weights of variance, skewness and kurtosis move further from their small random initial values and grow steadily in magnitude. The weight for entropy stays closest to zero throughout,

since its correlation is very low with target class. By epoch 50 the final weight vector was $[-0.1931, -0.2132, -0.2176, -0.0217]$.

8.7 Bias Evolution

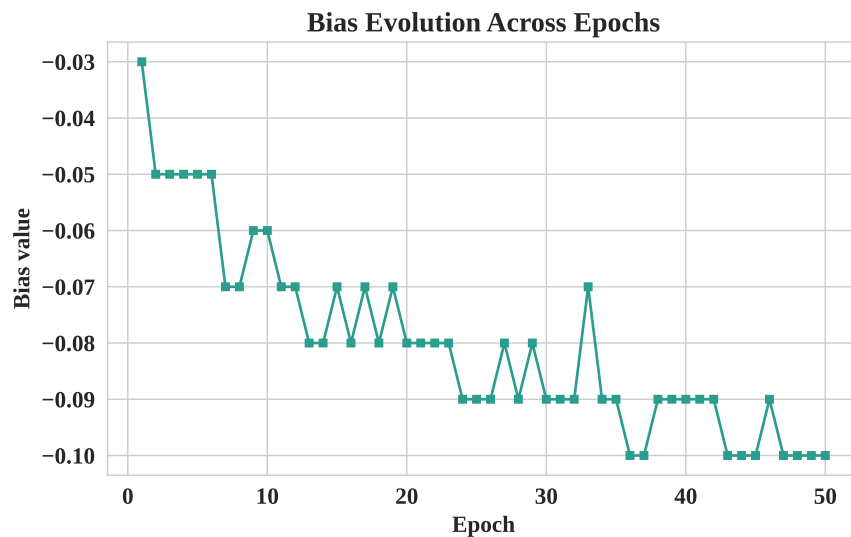


Figure 7: Evolution of the bias term across training epochs.

The bias term shifts steadily downward during training, from 0 initially to -0.10 by epoch 50, translating the decision hyperplane to better separate the two classes.

8.8 Confusion Matrix

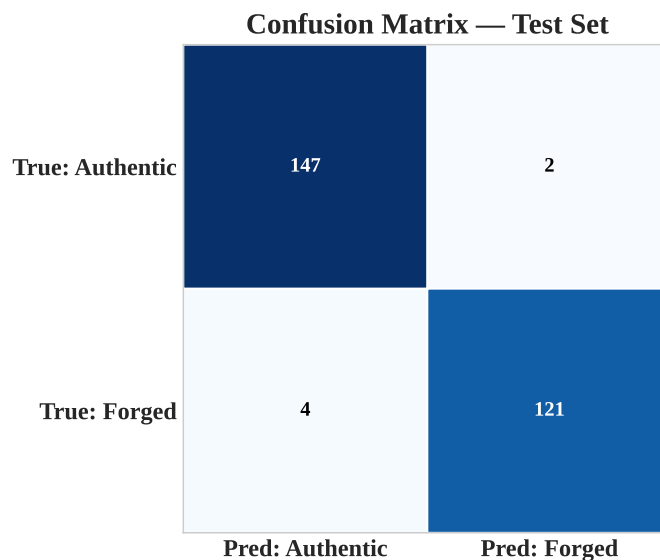


Figure 8: Confusion matrix on the held-out test set.

The model correctly classifies the large majority of test samples ($TP = 121$, $TN = 147$), with only 2 FP and 4 FN. These errors tend to be in the overlapping region of variance and skewness.

8.9 Learning Rate Comparison

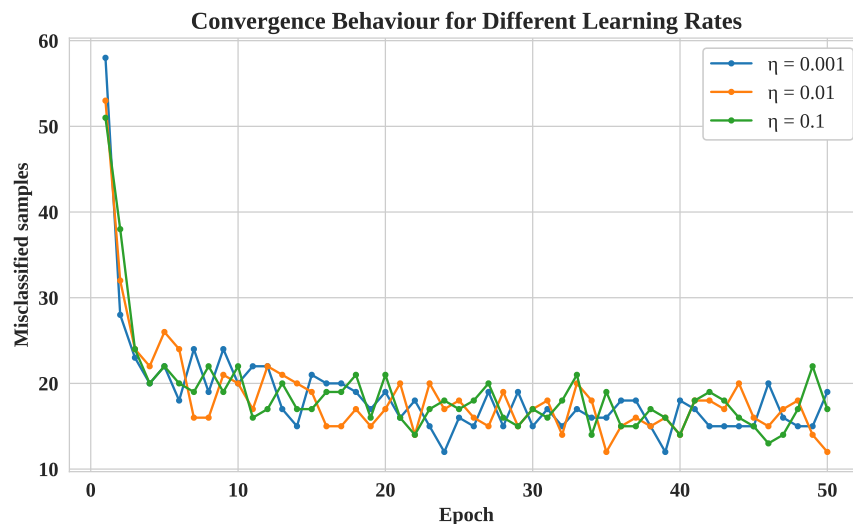


Figure 9: Convergence behaviour for $\eta \in \{0.001, 0.01, 0.1\}$.

$\eta = 0.001$ converges slowly and smoothly, taking many more epochs to reach a low error count. $\eta = 0.01$ balances speed and stability well. $\eta = 0.1$ converges fastest initially but shows more oscillation between epochs, since each weight update is larger and can overshoot the ideal boundary – a classic large-learning-rate trade-off.

8.10 Decision Boundary

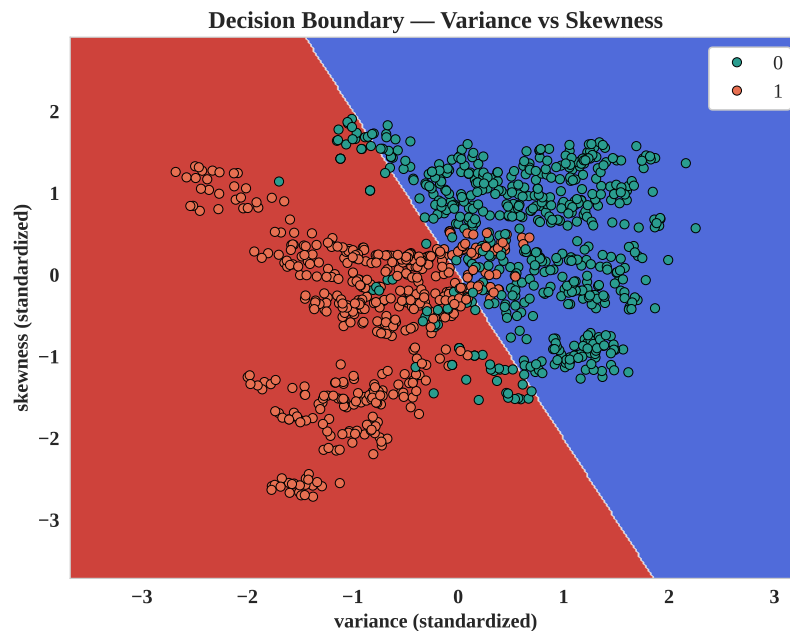


Figure 10: Decision boundary using variance and skewness only.

The shaded regions show the linear boundary the 2-feature perceptron has learned. Most points fall on the correct side of the line, but the visible misclassifications near the boundary show that the two classes are non-linearly separable using only these two features.

9. Performance Tables

Training Summary

Dataset Size	1372 (1097 train / 275 test)
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs	50 (ran to maximum; did not reach zero training error)
Final Weights	$[-0.1931, -0.2132, -0.2176, -0.0217]$
Final Bias	-0.1000
Accuracy	0.9781
Precision	0.9837
Recall	0.9680
F1-score	0.9758

Confusion Matrix (Test Set)

	Predicted Authentic	Predicted Forged
Actual Authentic	TN = 147	FP = 2
Actual Forged	FN = 4	TP = 121

Epoch-wise Learning (selected epochs)

Epoch	Errors	Bias	$\ \mathbf{w}\ $
1	53	-0.0300	0.1064
2	32	-0.0500	0.1425
3	24	-0.0500	0.1650
4	22	-0.0500	0.1785
5	26	-0.0500	0.1941
10	20	-0.0600	0.2321
20	17	-0.0800	0.2782
30	17	-0.0900	0.3136
40	14	-0.0900	0.3399
50	12	-0.1000	0.3613

Scikit-learn Comparison

Metric	Custom RosenblattUnit	sklearn.Perceptron
Accuracy	0.9781	0.9781
Precision	0.9837	0.9760
Recall	0.9680	0.9760
F1-score	0.9758	0.9760

10. Additional Tasks

10.1 Step vs Sigmoid Activation

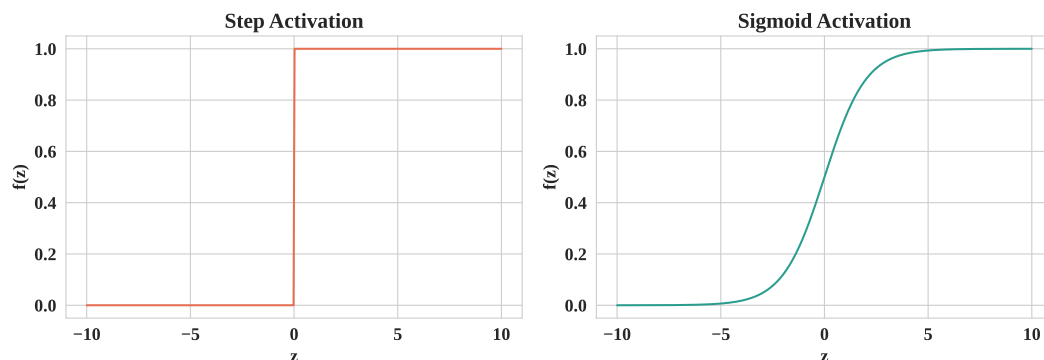


Figure 11: Step activation (left) vs. Sigmoid activation (right).

The step function shows a discontinuity at $z = 0$, flat at 0 till that and an instant jump to 1, with zero gradient everywhere except that single undefined point. The sigmoid curve is smooth and continuous across the same range, with a non-zero gradient throughout most of its domain.

10.2 Comparison with Scikit-learn's Perceptron

Both implementations achieve identical test accuracy (0.9781). The custom implementation has marginally higher precision (0.9837 vs. 0.9760) but slightly lower recall (0.9680 vs. 0.9760) than built-in perceptron, which makes the F1 score identical almost. This indicates that manual implementation is correct, with marginal variation with results.

10.3 Learning Rate Comparison

Learning rates of 0.001, 0.01, 0.1 were tested. Lower learning rates converge more slowly but smoothly, while higher learning rates converge faster initially but oscillate more. These oscillations are caused because sample wise decision boundaries are large, and will overshoot the decision boundary.

10.4 Why a Single Layer Perceptron Cannot Solve XOR

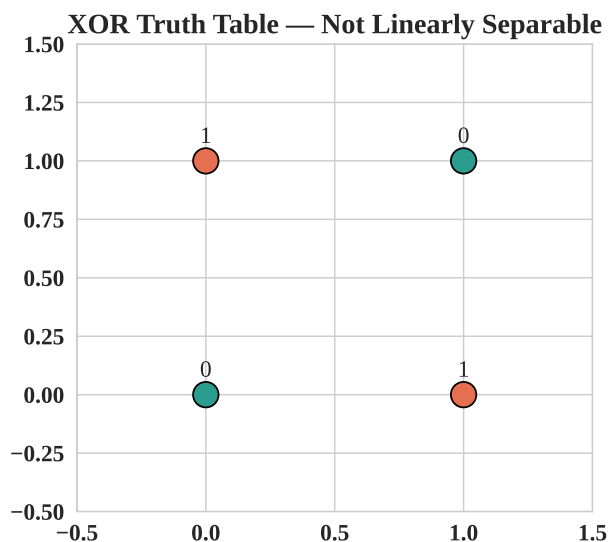


Figure 12: XOR truth table – not linearly separable.

The four XOR points form two classes positioned at diagonally opposite corners of the square. No single straight line can separate the two colors without misclassifying at least one point. This shows why single layer perceptron cannot solve XOR problem, as there is no linear separability.

10.5 Effect of Feature Normalization on Convergence

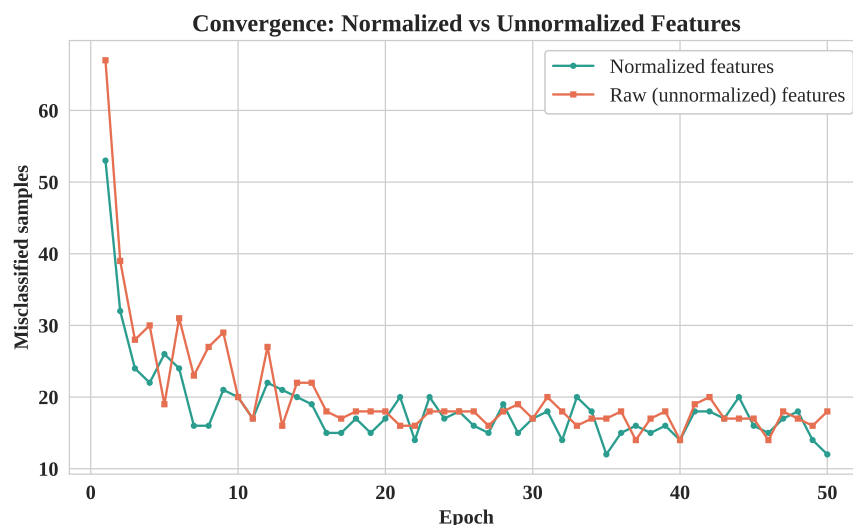


Figure 13: Convergence comparison: normalized vs. unnormalized features.

The normalized-feature curve converges faster and more smoothly than the unnormalized curve. Without normalization, features with larger natural numeric ranges like skewness, kurtosis dominate the weight updates, slowing convergence and causing more erratic error counts across epochs.

11. Additional Experiment

The perceptron learning algorithm was implemented independently for three fundamental logic gates (AND, OR, NOT) for 2 bit data to observe how a single layer perceptron learns different linearly separable boolean functions. Weights and biases were initialized as 0 for every gate, and the perceptron weight update algorithm was applied., and decision boundaries were plotted after the model converges. This experiment proves that single layer perceptron work well on linearly separable data, as there are no errors in the model post convergence and the model has classified all data points correctly.

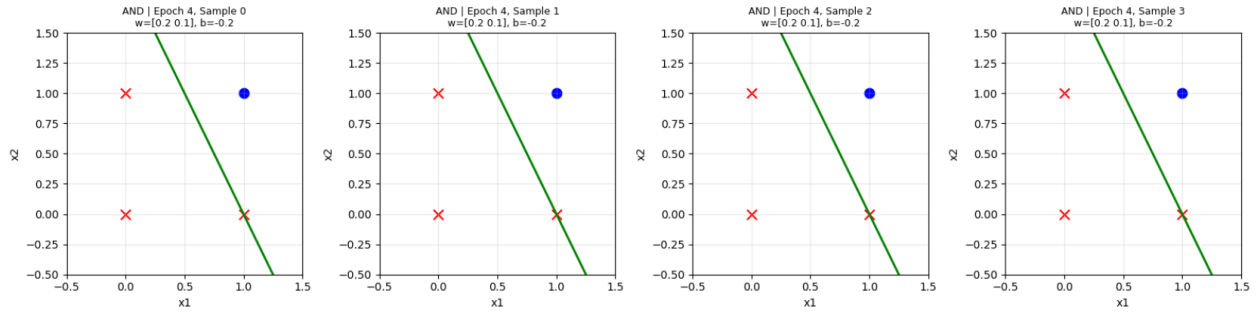


Figure 14: Decision boundary evolution for the AND gate.

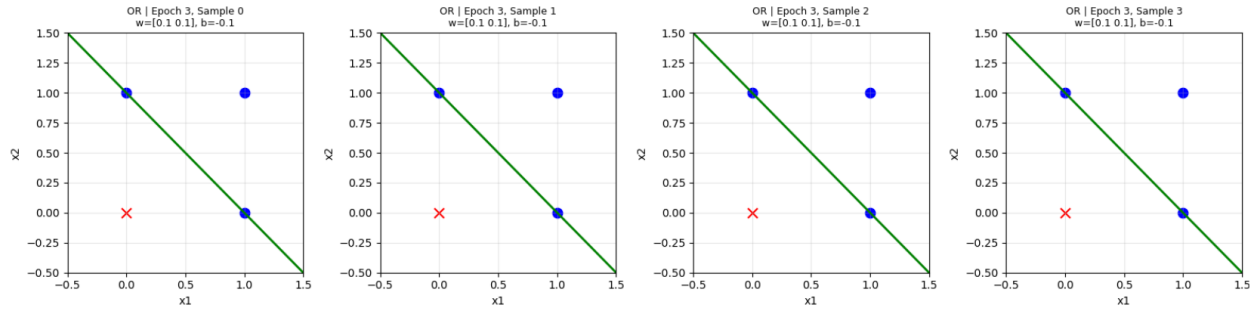


Figure 15: Decision boundary evolution for the OR gate.

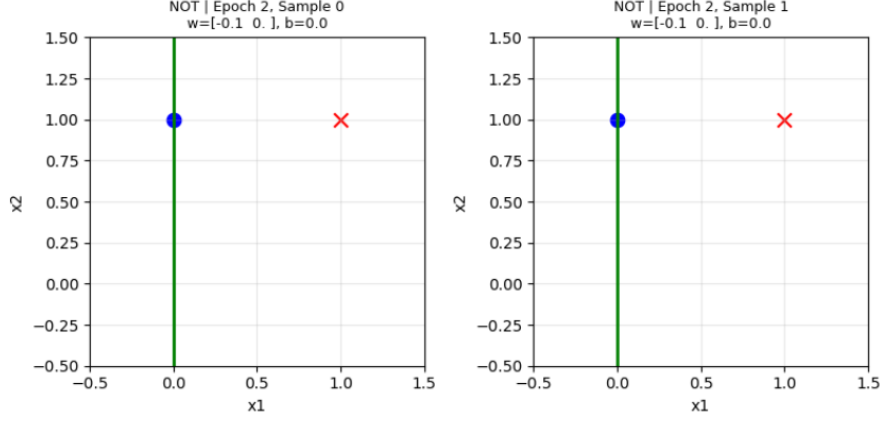


Figure 16: Decision boundary evolution for the NOT gate.

Initial and Final Weights/Bias Summary

Gate	Initial Weights	Initial Bias	Final Weights	Final Bias	Epochs Taken
AND	[0,0]	0	[0.2,0.1]	-0.2	4
OR	[0,0]	0	[0.1,0.1]	-0.1	3
NOT	[0,0]	0	[-0.1,0]	0	2

12. Discussion

The single-layer perceptron, trained from scratch using Rosenblatt’s original learning rule, achieved strong performance (97.81% test accuracy, F1-score 0.9758). This shows that the given features are highly discriminative between the two classes. Variance, skewness and curtosis contributed majorly in identifying the classes, while entropy had contributed the least.

The training-error curve shows the model did not reach zero misclassifications even after 50 epochs, and misclassified 12 data points on the training set. This indicates the classes are close to, but not perfectly linearly separable. A perceptron with a hard step activation cannot resolve this partial overlap even on training with more epochs, since it produces only a hyperplane as a decision boundary.

The learning rate study demonstrates bias-variance, i.e smaller η gives smoother but slower convergence, while larger η risks oscillation from overshooting the optimal decision boundary. The comparison against Scikit-learn’s builtin perceptron proves the correctness of the manual implementation from scratch as both achieve only marginally different performance (caused by minor implementation differences).

The single layer perceptron was then tried on basic logic gates of AND, OR, NOT and XOR. The model was able to produce clear decision boundaries for linearly separable data like AND, OR and NOT gate. However, it could not produce proper decision boundaries which classifies all points correctly, as there is no possible linear boundary for the XOR problem. This additional experiment clearly points out the flaw in single layer perceptrons, which is the lack of ability to draw decision boundaries for non-linearly problems. This problem can be solved by multilayer perceptrons with hyperparameter tunings, to be studied on upcoming experiments.

13. Conclusion

In this experiment, a Single Layer Perceptron was successfully implemented from scratch using NumPy and applied to the UCI Banknote Authentication dataset. The perceptron learning rule, step activation function, and weight and bias update mechanism were implemented manually. The trained model achieved 97.81% accuracy, 98.37% precision, 96.80% recall, and an F1-score of 0.9758 on the test set, which is almost as same as the builtin perceptron implementation.

The additional experiment demonstrated clearly how single layer perceptrons can solve only linearly separable problems, and why they struggle with non-linearly separable datasets. Other additional tasks compared the efficiency of step and sigmoid activation functions, tested multiple learning rates to find optimal value, and studied the effect of normalization on convergence.

14. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

15. Source Code

The complete source code, generated outputs, and result tables for this experiment are available at the following GitHub repository:

<https://github.com/Ashwin271206/dl-lab/tree/main/Lab%201>